سؤال ۱: (۱۶ نمره)

با ذکر دلیل مشخص کنید گزاره‌های زیر صحیح هستند یا خیر. به پاسخ بدون ذکر دلیل نمره‌ای تعلق نمی‌گیرد.

الف) مزیت حفظ سلسله‌مراتب طرح (design hierarchy) این است که نتیجهٔ ابزارها از نظر مساحت طرح بهتر خواهد شد.

ب) هزینهٔ غیرتکراری (NRE Cost) با افزایش تعداد تراشهٔ تولیدشده کم می‌شود.

ج) تراشه‌های مبتنی بر SRAM با هر بار روشن شدن باید مجدداً برنامه‌ریزی شوند، به همین دلیل از امنیت مناسبی برخوردار هستند.

د) تغییرات ناخواسته در سیگنال‌ها (گلیچ‌ها) موجب افزایش توان مصرفی ایستا می‌شوند.

سؤال ۲: (۲۰ نمره)

مدار زیر را در نظر بگیرید.

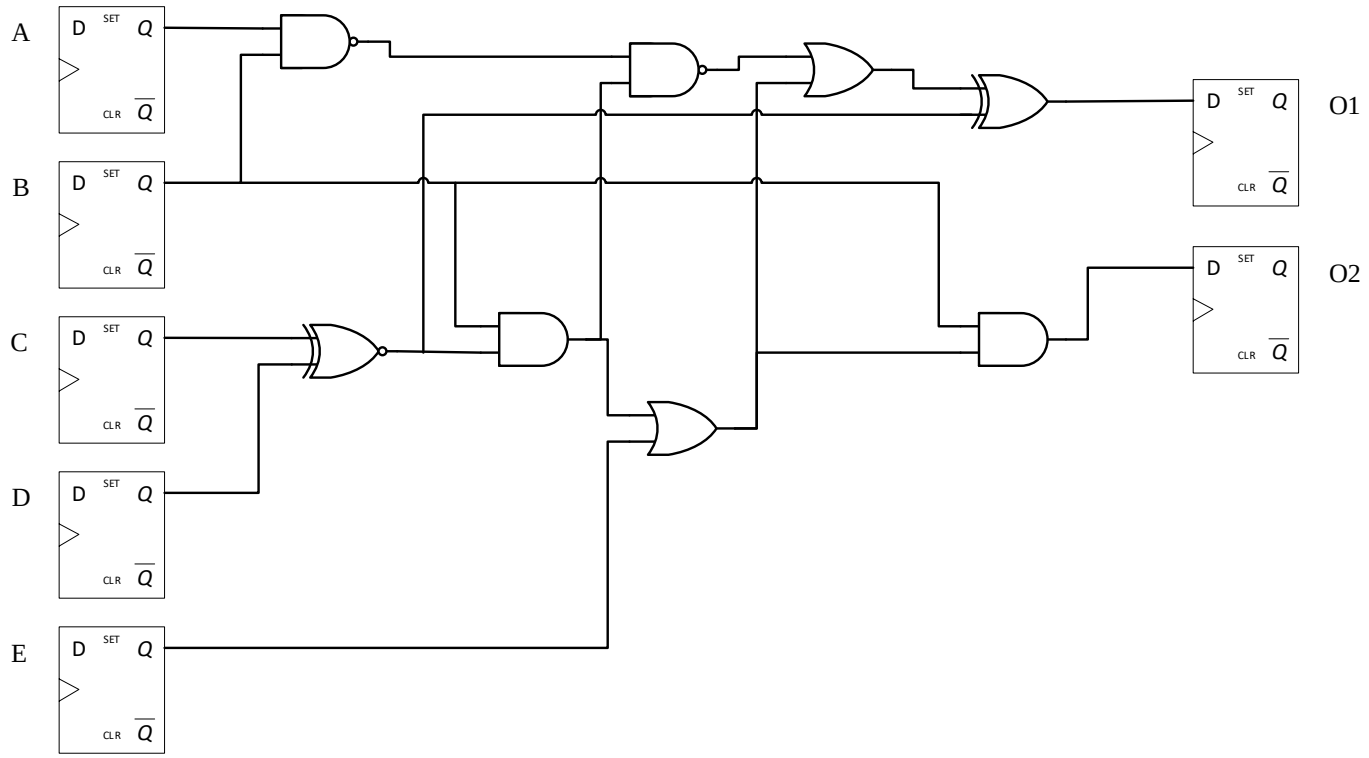
الف) گراف تأخیر مدار زیر را رسم کنید (تمامی درگاه‌های ورودی و خروجی دروازه‌های منطقی باید در مدل شما وجود داشته باشد).

ب) اکنون بر اساس گراف بخش (الف) فرکانس کاری مدار را محاسبه کنید. مقادیر زمان‌های راه‌اندازی (setup time)، نگهداشت (hold time) و انحراف ساعت (clock skew) و تاخیر ساعت به خروجی (clock-to-output) فلیپ‌فلاپ‌ها صفر است. در صورتی که بخواهیم سرعت مدار بهبود یابد، پیشنهاد شما بهبود کدام بخش از مدار است؟

ج) فرض کنید مقادیر setup و hold و تاخیر ساعت به خروجی فلیپ‌فلاپ‌ها به ترتیب ۲ و ۰ و ۱ نانوثانیه باشند. همچنین اگر سیگنال ساعت فلیپ‌فلاپ‌های ورودی نسبت به سیگنال ساعت فلیپ‌فلاپ‌های خروجی ۱ نانوثانیه دیرتر برسد فرکانس کاری مدار چه تغییری می‌کند؟

تاخیر دروازه‌های منطقی مطابق جدول زیر است. از تاخیر سیم‌ها صرف نظر کنید.

Gate	Delay
And/Or	5ns
Nand/Nor	3ns
Xor/Xnor	7ns



سؤال ۳: (۱۸ نمره)

مداری را که از توصیف زیر سنتز می‌شود، رسم کنید.

مدار را بر حسب بلوک‌های زیر رسم کنید: (ممکن است بعضی از این بلوک‌ها در مدار وجود نداشته باشد)

فلیپ‌فلاپ (اگر ثبات nبیتی تولید می‌شود، آن را به صورت n فلیپ‌فلاپ رسم کنید)، جمع‌کننده، مقایسه‌کننده، مالتی‌پلکسر و گیت‌های منطقی، لچ)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity pwm is
  port(
      clk, reset: in std_logic;
      w: in std_logic_vector(3 downto 0);
      pwm_pulse: out std_logic
      );
end pwm;

architecture two_seg_arch of pwm is
  signal r_reg: unsigned(3 downto 0);
  signal r_next: unsigned(3 downto 0);
  signal buf_reg: std_logic;
  signal buf_next: std_logic;
begin

process(clk, reset)
  variable v1: std_logic;
begin
  if (reset = ’1’) then
    r_reg <= (others => ’0’);
    buf_reg <= ’0’;
  elsif (clk’event and clk = ’1’) then
    r_reg <= r_next;
    v1 := buf_next;
    buf_reg <= v1;
  end if;
end process;

r_next <= r_reg + 1;

buf_next <=
  ’1’ when (r_reg < unsigned(w)) or (w="0000") else
  ’0’;
pwm_pulse <= buf_reg;
end two_seg_arch;
```

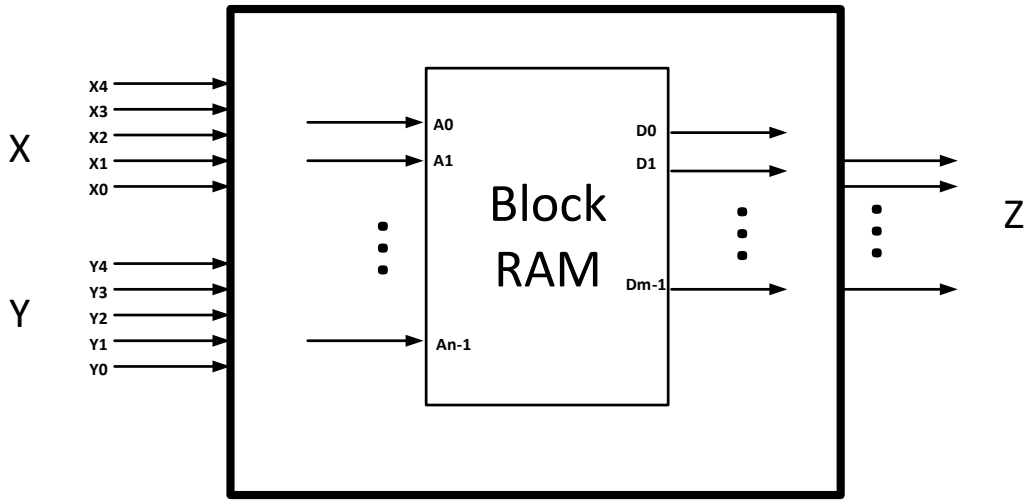
سؤال ۴: (۱۸ نمره)

الف) برای پیاده‌سازی یک ضرب‌کنندهٔ ۵ بیت در ۵ بیت با بلوک حافظهٔ درون FPGA، حداقل ابعاد حافظهٔ موردنیاز چیست؟ چرا؟

ب) محتوای دو خانهٔ اول، محتوای خانهٔ با آدرس ۱۰۱۰۰۰۱۱ و محتوای خانهٔ آخر این حافظه را به‌صورت باینری بنویسید. (در صورت نیاز به تعداد بیت بیشتر، در سمت چپ صفر در نظر بگیرید).

ج) بیت‌های عملوندهای X و Y و بیت‌های حاصل‌ضرب Z را به‌طور مناسبی به خطوط آدرس و دادهٔ حافظه در شکل زیر وصل کنید.

د) اگر ابعاد بلوک حافظهٔ موجود در FPGA به‌صورت ۸ × ۱۰۲۴ باشد، و دقت نتایج اهمیت بالایی نداشته باشد، بندهای (ب) و (ج) را مجدداً پاسخ دهید.

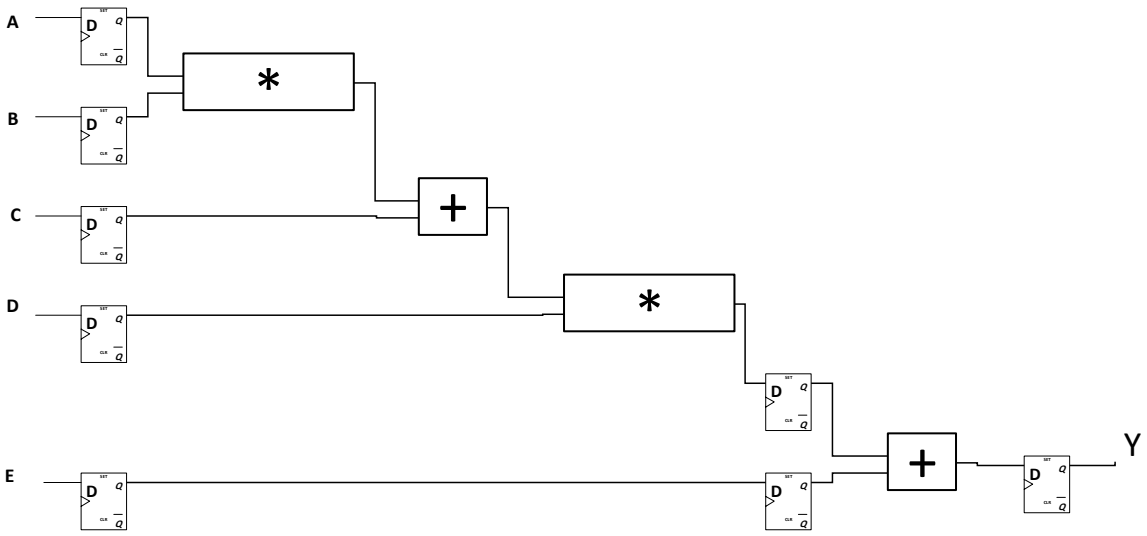


سؤال ۵: (۲۰ نمره)

الف) در مدار زیر، با فرض آنکه تأخیر ضرب‌کننده برابر با ۴۰ و تأخیر جمع‌کننده برابر با ۱۰ باشد، مدار را به گونه‌ای باززمان‌بندی (retime) کنید که فرکانس کار مدار بهبود یابد. تغییر در مدار را روی شکل نشان دهید. (latency مدار نباید تغییر کند). ثبات‌های ورودی و خروجی نباید تغییر کنند.

ب) مدار اولیه را در سطح RTL (با استفاده از پروسس و عملگرهای جمع و ضرب) توصیف کنید.

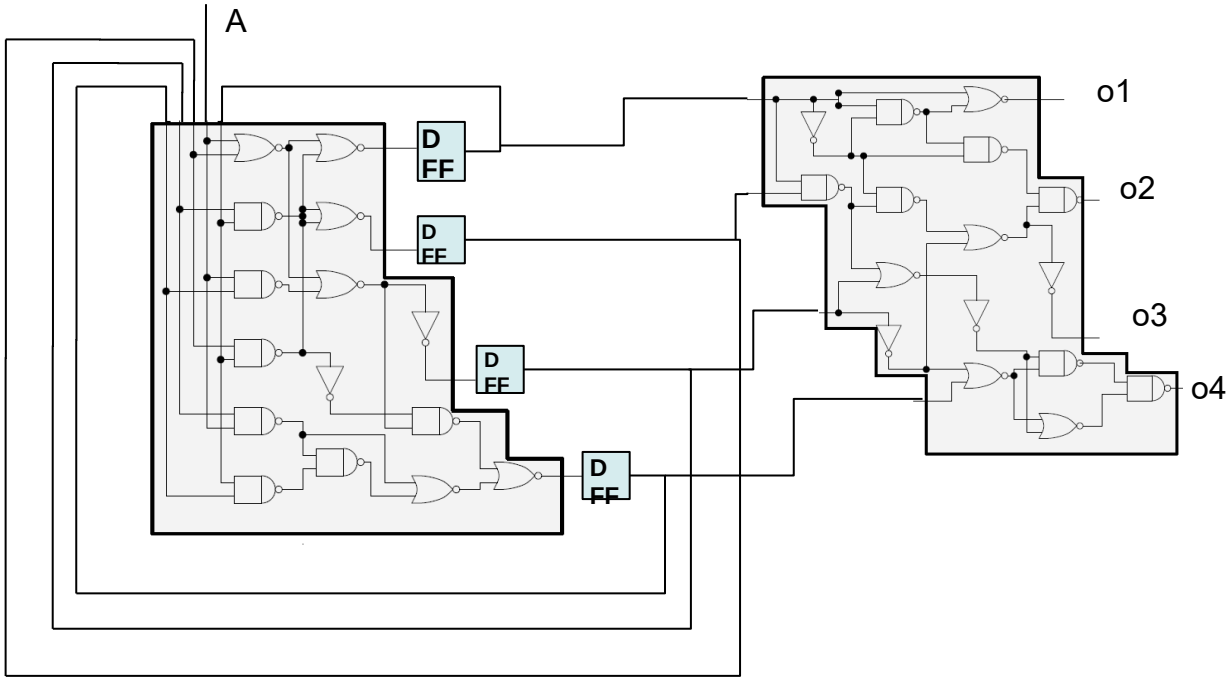
ج) مدار بعد از باززمان‌بندی را توصیف کنید.



سؤال ۶: (۲۰ نمره)

(۲۰ نمره) (الف) در مدار زیر با فرض آنکه تأخیر هر گیت ۱۰ نانوثانیه و تأخیر اتصالات ناچیز باشد، حداکثر فرکانس کلاک را تعیین کنید. زمان setup فلیپ فلاپ ها ۲ نانوثانیه، زمان تأخیر از کلاک تا خروجی فلیپ فلاپ ها (tco) ۱ نانوثانیه و زمان صعود و نزول کلاک (rise/fall time) ناچیز است. در ضمن همه فلیپ فلاپ‌ها کلاک را همزمان دریافت می‌کنند.

(ب) زمان آماده شدن خروجی کل مدار را (از آمدن لبه کلاک) حساب کنید.

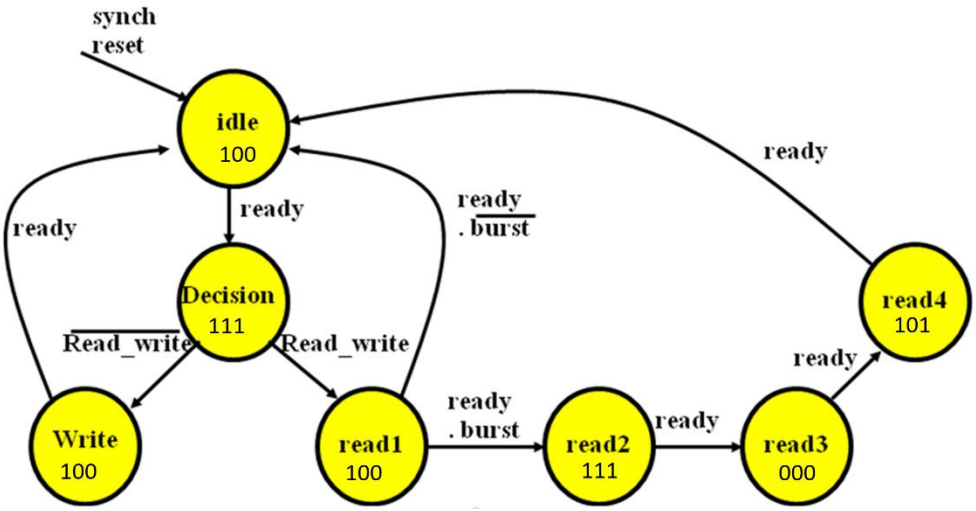


سؤال ۷: (۲۰ نمره)

الف) برای ماشین حالت زیر، انتساب کد به حالات را به‌گونه‌ای انجام دهید که خروجی‌ها مستقیماً از خروجی فلیپ فلاپ‌ها (حداکثر پس از تأخیر t_{co} (clock to output)) دریافت شوند.

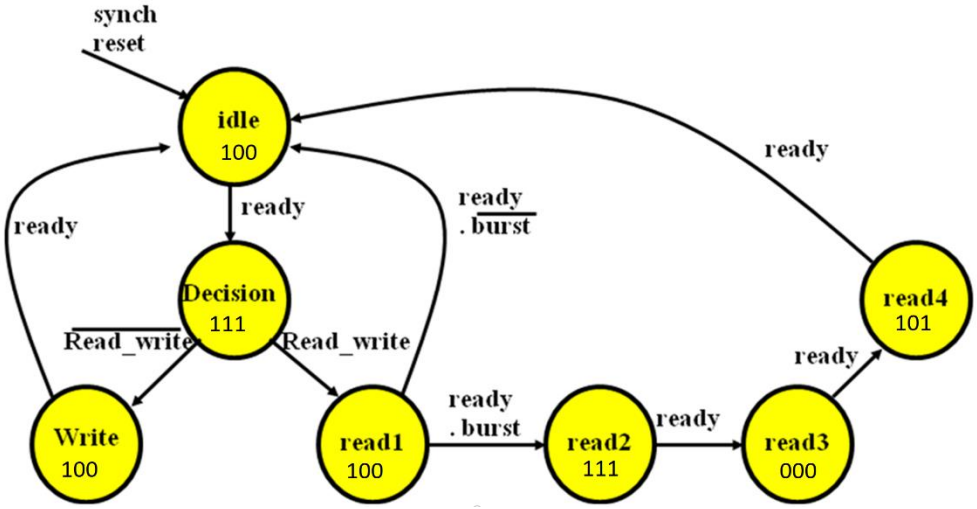
ب) قطعه‌ای از کد VHDL را که در آن انتساب حالت برای بخش (الف) انجام می‌شود، بنویسید.

ج) اگر بخواهیم انتساب کد را به **sequential** تغییر دهیم، با کمک **attribute**، دستورات تولید محدودیت (**constraint**) را در کد **VHDL** بنویسید.



سؤال ۸: (۲۰ نمره)

(۲۵ نمره) برای ماشین حالت زیر، توصیف VHDL با یک فرایند (process) بنویسید. توصیف شما باید کامل و شامل entity و architecture باشد. نوشتن کد برای سه حالت idle، Decision و Write کافی است ولی باید برای این سه حالت کامل باشد. برای سایر حالات می‌توانید از نقطه‌چین استفاده کنید. بازنشانی سیستم باید به صورت همگام با کلاک (synchronous reset) باشد.



ب) برای آنکه تولید خروجی به صورت ثبت شده باشند، تغییرات لازم در کد VHDL را بنویسید. توجه کنید که خروجی نباید نسبت به حالات، تأخیر یک پالس داشته باشد.

سؤال ۹: (۲۰ نمره)

(۲۵ نمره) در مدار شکل زیر می‌خواهیم خط لوله (pipeline) به کار بریم. در صورتی که تاخیر مربوط به هر جمع‌کننده، 2x (x واحد تاخیر فرض شود)، ضرب کننده، 4x و رجیستر برابر با x باشد و از تاخیرهای دیگر صرفنظر کنیم،

الف) در دو حالت بدون pipe و pipe با چهار مرحله زمان کل انجام کار برای ۱۰ سری عدد را حساب نمایید و رجیسترهای اضافه شده در حالت pipeline را در شکل نشان دهید. در همه موارد رجیسترهای ورودی و خروجی را به حساب نیاورید.

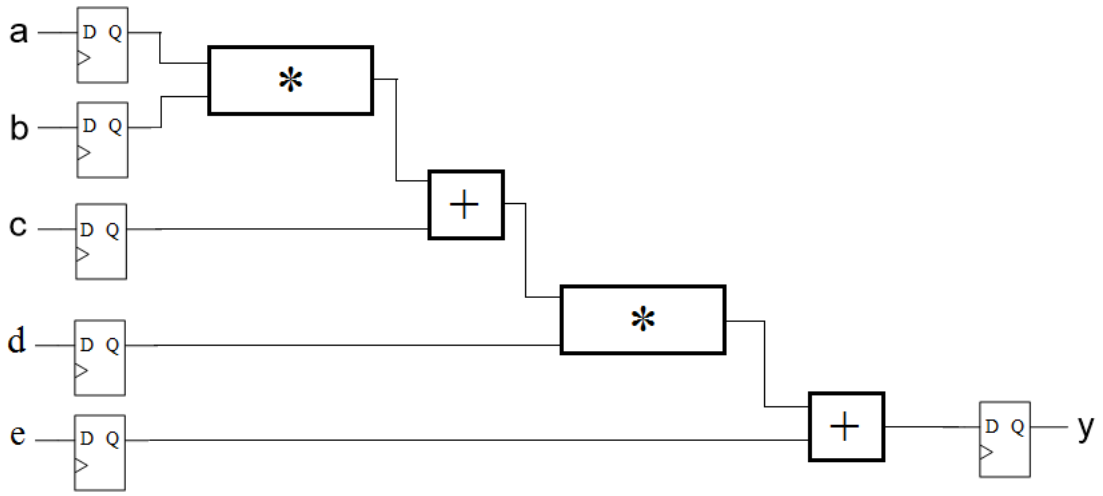
ب) دو دلیل نرسیدن به افزایش سرعت ۴ برابر در pipeline چهار مرحله ای را ذکر کنید.

ج) مساحت هر دو حالت را حساب کنید. (با فرض مساحت جمع کننده 2y، ضرب کننده 4y و رجیستر برابر با y و صرفنظر کردن از بقیه مساحت‌ها).

د) توصیف مدار بدون pipeline را در سطح RTL بنویسید (از عملگرهای * و + استفاده کنید). (به توصیف در سطح ساختاری نمره‌ای تعلق نخواهد گرفت).

ه) تغییرات لازم در کد را پس از ایجاد pipeline مشخص کنید. برای توصیف فلیپ‌فلاپ‌ها از فرایندی استفاده کنید که از دستور زیر برای لبهٔ کلاک استفاده می‌کند:

if (CLK = '1' and CLK'event) ...



سؤال ۱۰: (۵ نمره)

مداری را روی تراشه اسپارتان ۷ با شماره XC7S15 سنتز کرده‌ایم و گزارش ابزار سنتز نشان می‌دهد ۲۰٪ از sliceها مصرف شده است.

الف) تعداد LUTهاو فلیپ‌فلاپ‌های مصرفی چقدر بوده است.

ب) آیا ممکن است بعد از جایابی و مسیریابی، این تعداد تغییر کند؟ چرا؟

موفق باشید

پیوست:

VHDL Cheat-Sheet

©Copyright: 2007 Bryan J. Mealy

Concurrent Statements		Sequential Statements
Concurrent Signal Assignment	⇔	**Signal Assignment**
`target <= expression;`		`target <= expression;`
`A <= B AND C;` `DAT <= (D AND E) OR (F AND G);`		`A <= B AND C;` `DAT <= (D AND E) OR (F AND G);`
Conditional Signal Assignment	⇔	***if* statements**

`target <= expressn when condition else`
`expressn when condition else`
`expressn;`
		`if (condition) then`
		`{ sequence of statements }`
		`elsif (condition) then`
		`{ sequence of statements }`
		`else --(the else is optional)`
		`{ sequence of statements }`
		`end if;`

`F3 <= '1' when (L='0' AND M='0') else`
`'1' when (L='1' AND M='1') else`
`'0';`
		`if (SEL = "111") then F_CTRL <= D(7);`
		`elsif (SEL = "110") then F_CTRL <= D(6);`
		`elsif (SEL = "101") then F_CTRL <= D(1);`
		`elsif (SEL = "000") then F_CTRL <= D(0);`
		`else F_CTRL <= '0';`
		`end if;`

Selective Signal Assignment	⇔	***case* statements**

`with chooser_expression select`
`target <= expression when choices,`
`expression when choices;`
		`case (expression) is`
		`when choices =>`
		`{sequential statements}`
		`when choices =>`
		`{sequential statements}`
		`when others => -- (optional)`
		`{sequential statements}`
		`end case;`

`with SEL select`
`MX_OUT <= D3 when "11",`
`D2 when "10",`
`D1 when "01",`
`D0 when "00",`
`'0' when others;`
		`case ABC is`
		`when "100" => F_OUT <= '1';`
		`when "011" => F_OUT <= '1';`
		`when "111" => F_OUT <= '1';`
		`when others => F_OUT <= '0';`
		`end case;`

Process		

`opt_label: process(sensitivity_list)`
`begin`
`{sequential_statements}`
`end process opt_label;`		

`proc1: process(A,B,C)`
`begin`
`if (A = '1' and B = '0') then`
`F_OUT <= '1';`
`elsif (B = '1' and C = '1') then`
`F_OUT <= '1';`
`else`
`F_OUT <= '0';`
`end if;`
`end process proc1;`		

نمونه کدهای VHDL

```
entity FULLADDER is
   port (A,B, CARRY_IN: in   bit;
          SUM, CARRY:     out bit);
end FULLADDER;

architecture STRUCT of FULLADDER is
   signal W_SUM, W_CARRY1, W_CARRY2 : bit;
    component  HALFADDER
      port (A, B :              in   bit;
             SUM, CARRY : out bit);
    end component;
    component  ORGATE
      port (A, B : in   bit;
             RES : out bit);
    end component;
begin
    MODULE1: HALFADDER port map( A, B, W_SUM, W_CARRY1 );
    MODULE2: HALFADDER port map ( W_SUM, CARRY_IN, SUM, W_CARRY2 );
    MODULE3: ORGATE port map ( W_CARRY2, W_CARRY1, CARRY );
end STRUCT;

configuration CFG_FULLADDER of FULLADDER is
   for STRUCT
       for MODULE2: HA
          use entity
             work. HALFADDER(GATE);
       end for;

       for others : HA
          use entity
                work. HALFADDER(RTL);
       end for;
   end for;
end CFG_FULLADDER;

function COUNT1 (X: std_logic_vector (7 downto 0)) return integer is
   variable NoOfOnes : integer := 0;
begin
   for I in X'range loop
      if X(I) = '1' then
         NoOfOnes := NoOfOnes + 1;
      end if;
   end loop;
   return NoOfOnes;
end COUNT1;
```